

Doc. no. P0180R1
Date: 2016-03-21
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Reserve a New Library Namespace Future Standardization

Table of Contents

1. [Revision History](#)
 - [Revision 0](#)
2. [Introduction](#)
3. [Why We Need A New Namespace](#)
4. [Propose Solution](#)
 - [Is the problem urgent?](#)
 - [Choosing a namespace](#)
 - [Restrictions using a future namespace](#)
5. [Formal Wording](#)
6. [References](#)

Revision History

Revision 0

Original version of the paper for the 2016 pre-Jacksonville mailing.

Revision 1

Revisions based on Jacksonville review...

Introduction

Over the last few meetings, there have been informal suggestions that we should consider larger-scale changes for a revised standard library at some point in the future, and that time is closing. This paper suggests that we should reserve a namespace for that potential library now, so give the best chance that name will be free for use in standardization when needed.

Why We Need A New Namespace

As the number of changes to the core language accumulate, and a better understanding of how modern C++ libraries should be written to take work well with modern idioms grows, there are increasing demands to make more significant changes or revisions to the current standard library. At some point, maybe as early as C++20, we may want to have a library that clearly breaks backwards compatibility with the standard libraries that have gone before. Key pending features that might inspire more radical changes include the addition of concepts, concepts, implicit comparison operators (with a meaning different to that of the current library), unified function call syntax changing the relationship between member and free functions, and a desire to take advantage of more recent developments in the standard library, such as adopting generic (*transparent*) functors as the default library comparators. The Ranges TS gives a good insight into how a future library might be more capable, yet simpler to implement at the same time, if we can pick the appropriate abstractions. There are also several library issues that keep resurfacing in the concurrency clauses where it seems clear that we might have chosen a different behavior with but a little more feedback, but that are incredibly hard to change within the current standard library.

C++11 introduced the idea of `inline` namespaces to help solve the versioning problem. The key benefit of inline namespaces is that they allow a library vendor to preserve a stable ABI for old libraries, that existing code can continue to link against, while allowing new code to use the most recent version of the library, with an updated API, and potentially source-incompatible APIs.

The problem with relying on inline namespaces to solve the standard library versioning problem is that they did not exist when most of the current standard library implementations were created, so they are not in a position to use inline namespaces to preserve compatibility for their current implementations. If we were starting a new library, that solution would be baked into the initial deployment, and the problem of future compatibility could then be solved.

Proposed Solution

This paper proposes reserving a new namespace for future standardization, while not yet putting anything in there. This is similar to reserving namespace `posix` for C++11, [N2667](#).

The namespace should be reserved now so that any existing code currently using that namespace has notice and time to migrate, before there is a real problem interfering with the standard. Likewise, authors of new code are on notice that this namespace is reserved. The longer we wait to reserve such a namespace, the fewer reasonable names will be available.

Using a whole new namespace for a future library allows for an smoother transition for users, who can migrate code from one library to the other at a pace of their own choosing, while allowing the Library groups to be more aggressive in their approach to a *mostly* source compatible revised library.

Is the problem urgent?

It is clear (I think) that we do not need a new namespace for the anticipated C++17 standard library. However, the longer we take to reserve a library for our future needs, the harder it will be to choose a useful, practical, namespace without seriously disrupting some of our community. It is important to put the C++ community at large on notice that some part of the universal naming space should be considered reserved for future standards, and C++17 is our next, best shipping vehicle to make that happen. With that said, we still have several meetings before shipping a CD to research which specific name might be most appropriate and least disruptive - the main urgency is to agree whether or not this direction is desirable at all.

Choosing a namespace

The ideal namespace would be short, pronounceable, and meaningful. As such, it is most likely taken by a variety of other organizations. Nevertheless, this paper will proceed with `std2` as a simple placeholder for a name that should be chosen, with some careful research, by the Library Evolution Working Group.

Restrictions using a future namespace

While picking a new namespace may solve a wide variety of migration issues for the standard library, there will remain a few key classes and APIs that cannot migrate to a new namespace. These are the classes designed for interaction with the core language, and are mostly specified in clause 18. The most obvious cases are the low level exception hierarchy, `exception`, `bad_alloc`, `bad_array_new_length`, `bad_cast`, `bad_exception`, and `bad_typeid`. Note that exceptions derived from `logic_error` or `runtime_error` are safely in the library domain, and not referenced by the core language. Additional library classes that interact directly with the compiler include `exception_ptr`, `initializer_list`, and `typeinfo`, in addition to registration facilities like the new and terminate handlers.

Suggestions for a namespace

The library Evolution Working Group came up with two suggestions, that should be pursued in parallel. The first was that we should use a name that is not otherwise available to the user, either by using characters that would not otherwise be valid for an identifier (e.g., `c++`) or by recycling a keyword (e.g., `export`). This would require a complicit compiler, and should be pursued with the Evolution Working Group.

The second option was to brainstorm as many reasonable (or unreasonable) sounding names as we might consider, and then perform due diligence with various code-searching tools between now and the next meeting, to see which candidate names might still be reasonable to consider.

The shortlist on names supplied by brainstorming in LEWG was:

- `cpp`
- `cxx`
- `lib`
- `std2`
- `std\d*` [regex]
- `stdex`
- `stdlib`
- `ste`
- `stx`
- `std.*` [regex]
- `wg21`
- `xtd`

Formal Wording

Insert the following new subsection into clause 17:

17.6.4.2.3 Namespace `iso` [`namespace.iso`]

1. The behavior of a C++ program is undefined if it adds declarations or definitions to namespace `std2` or to a namespace within namespace `iso` unless otherwise specified.
2. [Note: The namespace `std2` is reserved for use by future revisions of this standard. - *end note*]

Insert the following rationale into the compatibility Annex C:

C.4.x Removed Deprecated Feature [`diff.cpp14.library`]

17.6.4.2.3

Change: New reserved namespace

Rationale: Reserve namespace for a revised standard library.

Effect on original feature: The global namespace `iso` is now reserved for standardization. Valid C++ 2014 code that uses a top-level namespace `std2` may be invalid in this International Standard.

Acknowledgements

Thanks to Titus Winters for volunteering to help manage the code searches.

References

- [N2667](#) Reserved Namespace for POSIX, Benjamin Kosnik, Nick Stoughton
- [N4532](#) Proposed wording for default comparisons, Jens Maurer
- [N4553](#) Working Draft for Concepts TS
- [N4560](#) Working Draft for Ranges TS
- [P0143R0](#) Wording for Modules, Gabriel Dos Reis
- [N3421](#) Making Operator Functors greater<>, Stephan T. Lavavej
- [N3451](#) `async` and `~future`, Herb Sutter